

Aube RAMASSAMY

Judia OUAZARI

Isabelle SOMPHONE

Corentin MONSALLIER



ART CONNECT REPORT - DATABASE 2

Course Code – TI603I

ART CONNECT REPORT - DATABASE 2	1
---------------------------------------	---

Step 1 : Understanding ArtConnect and Defining the Functional Scope	2
1. Exploring the provided application	2
2. Functional analysis.....	2
3. Static design	4
Step 2: Conceptual and Logical Modeling of the ArtConnect Database	4
1. Identify entities and attributes	4
2. Create a Conceptual Data Model (CDM).....	6
3. Create a Logical Data Model (LDM)	6
4. Create the database schema :.....	8
Step 3 : Database Implementation and Advanced Features	8
1. Create and populate the database	8
2. Views, indexes, and access rights	8
3. Triggers and stored programs	12
4. Transactions	16
Step 4: Integrating the Database into the ArtConnect Java Application	17

Step 1 : Understanding ArtConnect and Defining the Functional Scope

1. Exploring the provided application

ArtConnect Pro is a JavaFX application that manages a local art community. The project is layered into the presentation, service, DAO/persistence, and model layers. We can observe that the application uses in-memory data, because we will have to implement it through a connexion to our SQL database.

2. Functional analysis

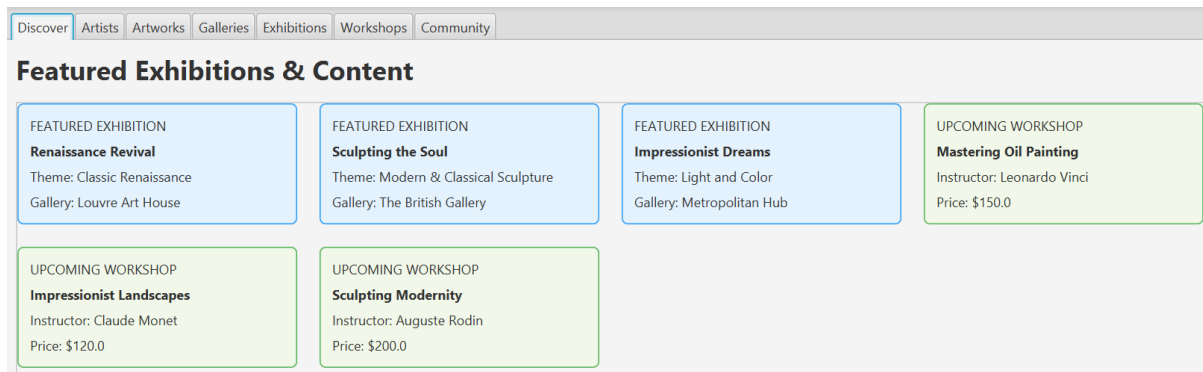


Figure 1 : Screenshot of a first visualization of the app

From the user perspective, the application provides the following features:

- Discover: view the exhibitions and upcoming workshops on a dashboard
- Artists: browse the full list of artists with their name, city, email and birth year
- Artworks: browse the list of artworks with title, artist, type, price and sale status
- Galleries: browse the list of galleries with address and rating
- Exhibitions: view the list of future exhibitions with the title, the host gallery, start date and theme
- Workshops: browse workshops with their title, instructor, date, price and level
- Community: view community members with their name, email and city

Primary Actors	Secondary actor
- Organizer: creates and manages exhibitions - Member of the community (registered user): browses content, can authenticate, book workshops, write reviews - Viewer (unregistered user): can browse and purchase artworks (must register for that) - Artists (potential instructor): uploads and manages their artworks and profile - Instructors: specialized Artist who leads workshops	Gallery: place that hosts exhibitions Bank: handles payment processing when a viewer buys an artwork



Figure 2 : Use-case diagram

3. Static design

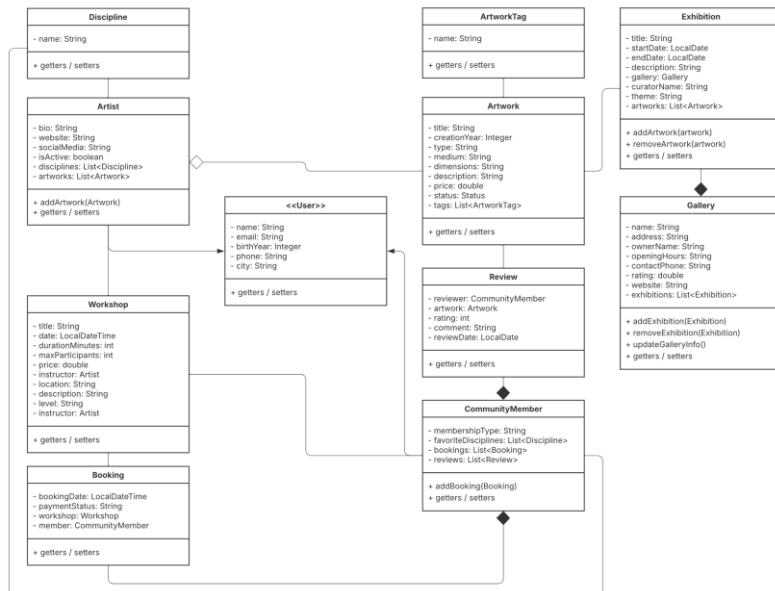


Figure 3 : Class diagrams

This class diagram provides an overview of the main entities of the application and the relationships that we want to implement. The class diagram highlights the object-

oriented structure of ArtConnect through different UML relationships such as associations and aggregations. This helps define how entities interact and depend on each other.

Step 2: Conceptual and Logical Modeling of the ArtConnect Database

1. Identify entities and attributes

Entities	Attributes	Dependencies	identifiers
User	Id_user Name Email BirthYear Phone City	Id_user <- Name, Email, BirthYear, Phone, City	Id_user
Artist	Id_artist Bio Website SocialMedia IsActive Disciplines artworks	Id_artist <- Bio, Website, SocialMedia, IsActive, Disciplines, Artworks	Id_artist
Discipline	Id_discipline name	Id_discipline <- name	Id_discipline
Workshop	Id_workshop Title Date DurationMinutes MaxParticipants Price Instructor Location Description Level	Id_workshop <- Title, Date, DurationMinutes, MaxParticipants, Price, Instructor, Location, Description, Level, Instructor Date <- DurationMinutes Description <- MaxParticipants, Price, location, level	Id_workshop
Community member	Id_Member MembershipType FavoriteDiscipline Bookings Review	Id_Member <- MembershipType, FavoriteDiscipline, Bookings, Review	Id_Member
Artwork	Id_Artwork Title CreationYear Type Medium Dimensions	Id_Artwork <- Title, CreationYear, Type, Medium, Dimensions, Description, Price, Status, Tags	Id_Artwork

	Description Price Status Tags	Type <- Medium, Dimensions, Description	
Booking	Id_Booking BookingDate PaymentStatus Workshop Member	Id_Booking <- BookingDate, PaymentStatus, Workshop, Member	Id_booking
Review	Id_Review Reviewer Artwork Rating Comment ReviewDate	Id_Review <- Reviewer, Artwork, Rating, Comment, ReviewDate	Id_Review
ArtworkTag	Id_Arttag Name	Id_Arttag <- Name	Id_Arttag
Exhibition	Id_Exhib Title StartDate EndDate Description Gallery CuratorName Theme artworks	Id_exhib <- title, StartDate, EndDate, Description, Gallery, CuratorName, Theme, Artworks	Id_Exhib
Gallery	Id_Gallery Name Address OwnerName OpeningHours ContactPhone Rating Website exhibitions	Id_Gallery <-Name, Address, OwnerName, OpeningHours, ContactPhone, Rating, Website, Exhibitions	Id_Gallery

2. Create a Conceptual Data Model (CDM)

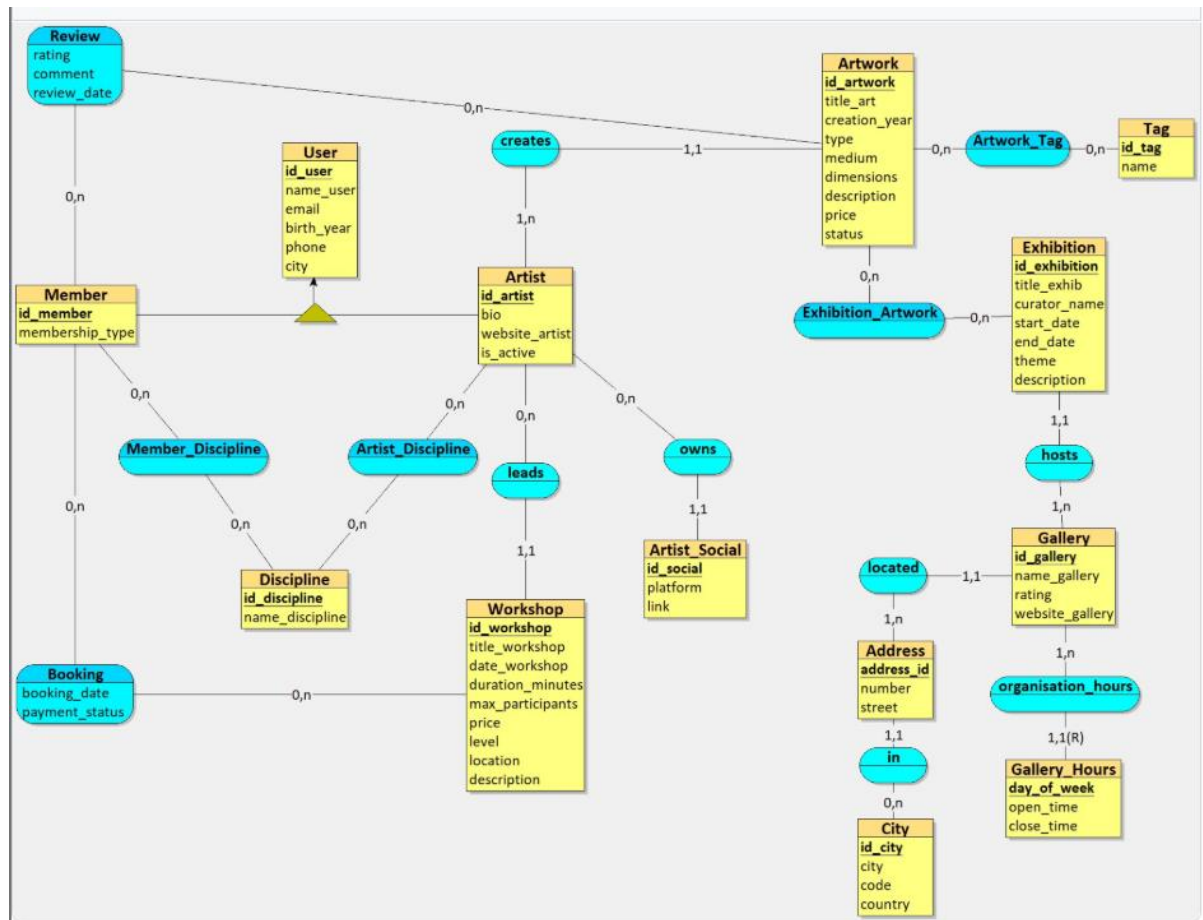


Figure 4 : CDM

On the CDM we can see between the user, artist and member a triangle. This specific triangle represents inheritance which means that the two subclasses that are Member and Artist inherit the attributes and methods of the superclass here User. We used this to explain that any artist and member are at first users, and the separation is only a differentiation with the information available about them and what they can do on our application.

3. Create a Logical Data Model (LDM)

User = (id_user INT, name_user VARCHAR(50), email VARCHAR(50), birth_year INT, phone VARCHAR(20), city VARCHAR(50));

Artist = (id_artist INT, bio TEXT, website_artist VARCHAR(100), is_active LOGICAL, #id_user);

Member = (id_member INT, membership_type VARCHAR(50), #id_user);

Discipline = (id_discipline INT, name_discipline VARCHAR(50));

Artwork = (id_artwork INT, title_art VARCHAR(100), creation_year INT, type VARCHAR(50), medium VARCHAR(50), dimensions VARCHAR(50), description TEXT, price DECIMAL(10,2), status ENUM('available', 'sold', 'reserved'), #id_artist);

Tag = (**id_tag** *INT*, name *VARCHAR(50)*);

Workshop = (**id_workshop** *INT*, title_workshop *VARCHAR(100)*, date_workshop *DATETIME*, duration_minutes *INT*, max_participants *INT*, price *DECIMAL(10,2)*, level *VARCHAR(50)*, location *VARCHAR(50)*, description *TEXT*, [#id_artist](#));

Artist_Social = (**id_social** *INT*, platform *VARCHAR(50)*, link *VARCHAR(200)*, [#id_artist](#));

City = (**id_city** *INT*, city *VARCHAR(50)*, code *INT*, country *VARCHAR(50)*);

Address = (**address_id** *INT*, number *INT*, street *VARCHAR(50)*, [#id_city](#));

Gallery = (**id_gallery** *INT*, name_gallery *VARCHAR(50)*, rating *INT*, website_gallery *VARCHAR(100)*, [#address_id](#));

Gallery_Hours = ([#id_gallery](#), **day_of_week** *VARCHAR(20)*, open_time *TIME*, close_time *TIME*);

Exhibition = (**id_exhibition** *INT*, title_exhib *VARCHAR(100)*, curator_name *VARCHAR(50)*, start_date *DATE*, end_date *DATE*, theme *VARCHAR(50)*, description *TEXT*, [#id_gallery](#));

Artist_Discipline = ([#id_artist](#), [#id_discipline](#));

Artwork_Tag = ([#id_artwork](#), [#id_tag](#));

Exhibition_Artwork = ([#id_artwork](#), [#id_exhibition](#));

Member_Discipline = ([#id_member](#), [#id_discipline](#));

Review = ([#id_member](#), [#id_artwork](#), rating *INT*, comment *TEXT*, review_date *DATETIME*);

Booking = ([#id_member](#), [#id_workshop](#), booking_date *DATETIME*, payment_status *ENUM('pending', 'paid', 'cancelled')*);

Classes like **Gallery_Hours** and **Artist_Social** were added to respect the First Normal Form 1NF, where each field must contain only one value and no repeating groups. Then, the database was normalized to the Third Normal Form (3NF) to reduce redundancy and improve data consistency. To normalize, we make sure that all attributes depend on the primary key and not on not key attribute.

4. Create the database schema :

All the SQL script are on our file Database.sql with the creation of tables, constraints and relationship .

Step 3 : Database Implementation and Advanced Features

1. Create and populate the database

When giving the files to ChatGPT : “According to these files, please fill each table of the database with at least ten rows.”

2. Views, indexes, and access rights

Goal	View's code
Artwork by artist	<pre>CREATE VIEW V_Artwork_By_Artist AS SELECT a.id_artist, u.name_user AS artist_name, aw.id_artwork, aw.title_art, aw.creation_year, aw.type, aw.price, aw.status FROM Artwork aw JOIN Artist a ON aw.id_artist = a.id_artist JOIN User_ u ON a.id_user = u.id_user ORDER BY a.id_artist, aw.id_artwork;</pre>
Artworks by discipline	<pre>CREATE VIEW V_Artworks_By_Discipline AS SELECT d.id_discipline, d.name_discipline, a.id_artwork, a.title_art FROM Discipline d JOIN Artist_Discipline ad ON d.id_discipline = ad.id_discipline JOIN Artist ar ON ad.id_artist = ar.id_artist JOIN Artwork a ON ar.id_artist = a.id_artist;</pre>
Artworks by review	<pre>CREATE VIEW V_Artwork_By_Review AS SELECT a.id_artwork, a.title_art, a.creation_year, a.type, a.medium, a.dimensions, a.description, a.price, a.status, r.rating, r.comment, r.review_date, r.id_member,</pre>

	<pre> u.name_user FROM Review r JOIN Artwork a ON r.id_artwork = a.id_artwork JOIN Member_ m ON r.id_member = m.id_member JOIN User_ u ON m.id_user = u.id_user ORDER BY r.rating; </pre>
Average price of the artworks of an artist	<pre> CREATE VIEW V_Avg_Artwork_Price AS SELECT a.id_artist, u.name_user AS artist_name, AVG(aw.price) AS avg_price, COUNT(aw.id_artwork) AS artwork_count FROM Artist a JOIN User_ u ON a.id_user = u.id_user JOIN Artwork aw ON aw.id_artist = a.id_artist GROUP BY a.id_artist ORDER BY a.id_artist; </pre>
Reviews by artwork	<pre> CREATE VIEW V_Review_By_Artwork AS SELECT a.id_artwork, a.title_art, a.creation_year, a.type, a.medium, a.dimensions, a.description, a.price, a.status, r.rating, r.comment, r.review_date, r.id_member, u.name_user FROM Review r JOIN Artwork a ON r.id_artwork = a.id_artwork JOIN Member_ m ON r.id_member = m.id_member JOIN User_ u ON m.id_user = u.id_user; </pre>
Workshop by availability (booking)	<pre> CREATE VIEW V_Workshop_Availability AS SELECT w.id_workshop, w.title_workshop, w.max_participants, COUNT(b.id_member) AS total_bookings, (w.max_participants - COUNT(b.id_member)) AS remaining_spots, CASE WHEN COUNT(b.id_member) < w.max_participants THEN 'available' ELSE 'full' END AS availability_status </pre>

	<pre> FROM Workshop w JOIN Booking b ON w.id_workshop = b.id_workshop AND b.payment_status <> 'cancelled' GROUP BY w.id_workshop, w.title_workshop, w.max_participants; </pre>
Members in workshop	<pre> CREATE VIEW V_Workshop_Participants AS SELECT w.id_workshop, w.title_workshop, m.id_member, u.name_user AS participant_name, b.payment_status, b.booking_date FROM Booking b JOIN Workshop w ON b.id_workshop = w.id_workshop JOIN Member_ m ON b.id_member = m.id_member JOIN User_ u ON m.id_user = u.id_user WHERE b.payment_status <> 'cancelled'; </pre>
Count workshops done already by an artist	<pre> CREATE VIEW V_Artist_Workshop_Count AS SELECT a.id_artist, u.name_user AS artist_name, COUNT(w.id_workshop) AS past_workshops FROM Artist a JOIN User_ u ON a.id_user = u.id_user JOIN Workshop w ON a.id_artist = w.id_artist AND w.date_workshop < NOW() GROUP BY id_artist, u.name_user; </pre>
Artist by exhibition	<pre> CREATE VIEW V_Artist_By_Exhibition AS SELECT e.id_exhibition, e.title_exhib, a.id_artist, u.name_user AS artist_name, COUNT(ea.id_artwork) AS artworks_in_exhibition FROM Exhibition e JOIN Exhibition_Artwork ea ON e.id_exhibition = ea.id_exhibition JOIN Artwork aw ON ea.id_artwork = aw.id_artwork JOIN Artist a ON aw.id_artist = a.id_artist JOIN User_ u ON a.id_user = u.id_user GROUP BY e.id_exhibition, a.id_artist ORDER BY e.id_exhibition, a.id_artist; </pre>

Available exhibition by gallery	<pre>CREATE VIEW V_Available_Exhibitions_By_Gallery AS SELECT g.id_gallery, g.name_gallery, e.id_exhibition, e.title_exhib, e.start_date, e.end_date, e.theme FROM Exhibition e JOIN Gallery g ON e.id_gallery = g.id_gallery WHERE CURDATE() BETWEEN e.start_date AND e.end_date;</pre>
Artist associated with their social(s)	<pre>CREATE VIEW V_Artist_Social AS SELECT a.id_artist, u.name_user AS artist_name, s.platform, s.link AS social_link FROM Artist a JOIN User_ u ON a.id_user = u.id_user JOIN Artist_Social s ON a.id_artist = s.id_artist;</pre>
Regroup info on users	<pre>CREATE VIEW V_User_Info AS SELECT u.id_user, u.name_user, u.email, u.birth_year, u.phone, u.city, m.membership_type, m.id_member FROM User_ u JOIN Member_ m ON u.id_user = m.id_user;</pre>

Goal	Index's code
Artwork	<pre>CREATE INDEX idx_artwork ON Artwork(id_artwork);</pre>
Artists	<pre>CREATE INDEX idx_artist ON Artist(id_artist);</pre>
Member	<pre>CREATE INDEX idx_member ON Member_(id_member);</pre>

3. Triggers and stored programs

Goal	Trigger's code
------	----------------

Rating between 1 and 5	<pre> DELIMITER // CREATE TRIGGER <i>rating_range</i> BEFORE INSERT ON review FOR EACH ROW BEGIN IF NEW.rating > 5 OR NEW.rating < 1 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: rating must be between 1 and 5!'; END IF; END; // DELIMITER ; </pre>
End date >= Start date	<pre> DELIMITER // CREATE TRIGGER <i>start_end_date</i> BEFORE INSERT ON Exhibition FOR EACH ROW BEGIN IF NEW.end_date < NEW.start_date THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: the end date must not be before the start date!'; END IF; END; // DELIMITER ; </pre>
Booking possible only if max_participant not reach	<pre> DELIMITER // CREATE TRIGGER <i>available_booking</i> BEFORE INSERT ON Booking FOR EACH ROW BEGIN DECLARE max_capacity INT; DECLARE current_bookings INT; -- max capacity of the workshop SELECT max_participants INTO max_capacity FROM Workshop WHERE id_workshop = NEW.id_workshop; -- current booked participants SELECT COUNT(*) INTO current_bookings FROM Booking WHERE id_workshop = NEW.id_workshop AND payment_status != 'cancelled'; IF current_bookings >= max_capacity THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: no available places left for this workshop!'; END IF; </pre>

	END; //
	DELIMITER ;

Goal	Function's code
Return the number of participant for a workshop	<pre> DELIMITER // CREATE FUNCTION F_Nb_Participants_Count(p_id_workshop INT) RETURNS INT DETERMINISTIC BEGIN DECLARE f_total INT; SELECT COUNT(*) INTO f_total FROM Booking WHERE id_workshop = p_id_workshop AND payment_status != 'cancelled'; RETURN f_total; END // DELIMITER ; </pre>

Goal	Stored procedure's code
Creation of a workshop	<pre> DELIMITER // CREATE PROCEDURE Create_Workshop (IN p_title VARCHAR(100), IN p_date DATETIME, IN p_duration INT, IN p_max_participants INT, IN p_price DECIMAL(10,2), IN p_level VARCHAR(50), IN p_location VARCHAR(50), IN p_description TEXT, IN p_id_artist INT) BEGIN INSERT INTO Workshop (title_workshop, date_workshop, duration_minutes, max_participants, price, level, location, description, id_artist) VALUES (</pre>

	<pre> p_title, p_date, p_duration, p_max_participants, p_price, p_level, p_location, p_description, p_id_artist); END // DELIMITER ; </pre>
Creation of an exhibition	<pre> DELIMITER // CREATE PROCEDURE Create_Exhibition (IN p_title_exhib VARCHAR(100), IN p_curator_name VARCHAR(50), IN p_start_date DATE, IN p_end_date DATE, IN p_theme VARCHAR(50), IN p_description TEXT, IN p_id_gallery INT) BEGIN INSERT INTO Exhibition (title_exhib, curator_name, start_date, end_date, theme, description, id_gallery) VALUES (p_title_exhib, p_curator_name, p_start_date, p_end_date, p_theme, p_description, p_id_gallery); END // DELIMITER ; </pre>
Count remaining spots in workshop	<pre> DELIMITER // CREATE PROCEDURE Check_Workshop_Capacity(IN p_id_workshop INT) BEGIN DECLARE p_max INT; </pre>

	<pre> DECLARE p_current INT; -- max capacity of the workshop SELECT COUNT(*) INTO p_current FROM Booking WHERE id_workshop = p_id_workshop AND payment_status != 'cancelled'; -- current booked participants SELECT COUNT(*) INTO p_current FROM Booking WHERE id_workshop = p_id_workshop AND payment_status != 'cancelled'; -- remaining spots SELECT p_max AS Capacity, p_current AS Booked, (p_max - p_current) AS Spots_Left; END // DELIMITER ; </pre>
Register a member in a workshop	<pre> DELIMITER // CREATE PROCEDURE Register_Member_To_Workshop(IN p_id_member INT, IN p_id_workshop INT) BEGIN DECLARE p_current_participants INT; DECLARE p_max_places INT; -- current booked participants SELECT COUNT(*) INTO p_current_participants FROM Booking WHERE id_workshop = p_id_workshop AND payment_status != 'cancelled'; -- maximum participants SELECT max_participants INTO p_max_places FROM Workshop WHERE id_workshop = p_id_workshop; IF p_current_participants < p_max_places THEN INSERT INTO Booking(id_member, id_workshop, booking_date, payment_status) VALUES(p_id_member, p_id_workshop, NOW(), 'pending'); ELSE SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Operation Error: no available places left for this workshop!'; END IF; END // DELIMITER ; </pre>

	<pre>END IF; END // DELIMITER ;</pre>
--	---

4. Transactions

Goal	Transaction's code
Register a user to multiple workshops	<pre> DELIMITER // CREATE PROCEDURE Register_To_Multiple_Workshops(IN p_id_member INT, IN p_id_workshop1 INT, IN p_id_workshop2 INT) BEGIN DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; END; START TRANSACTION; CALL Register_Member_To_Workshop(p_id_member, p_id_workshop1); CALL Register_Member_To_Workshop(p_id_member, p_id_workshop2); COMMIT; END // DELIMITER ; </pre>

Goal	Transaction's test code
This code was used to test whether the transaction works well or not	<pre> -- ===== -- TEST 1: SUCCESSFUL TRANSACTION -- ===== -- 1. Check bookings for member 5 SELECT * FROM Booking WHERE id_member = 5; -- 2. Execute the workshop registration CALL Register_To_Multiple_Workshops(5, 1, 2); -- 3. Verify that bookings exist SELECT * FROM Booking WHERE id_member = 5; -- ===== -- TEST 2: FAILURE & ROLLBACK -- ===== -- 1. Check bookings for member 6 SELECT * FROM Booking WHERE id_member = 6; -- 2. Execute the workshop registration CALL Register_To_Multiple_Workshops(6, 3, 999); -- 3. Verify that bookings don't exist SELECT * FROM Booking WHERE id_member = 6; </pre>

Step 4: Integrating the Database into the ArtConnect Java Application

To show the chosen architecture let explain by using a layer schema :

Database layer: All tables and information are placed in this layer to be used by our application later. All classes talked about in our LDM to normalized are implemented as models in java inside our code.

DAO layer : This layer is composed of files connecting the database with possible action like headers that will be implemented later in our code on another layer. Those files are created based on the most important classes of our project and initialize the action of selecting, inserting, deleting and updating the chosen class.

Service layer : Here we take the headers and link created on the dao layer to implement and specify the possible actions initiated. Again, we create those files based on the most important classes which are the same then what was made on the dao layer.

UI layer: Finally, this layer proposes a user view of the database directly on the application.

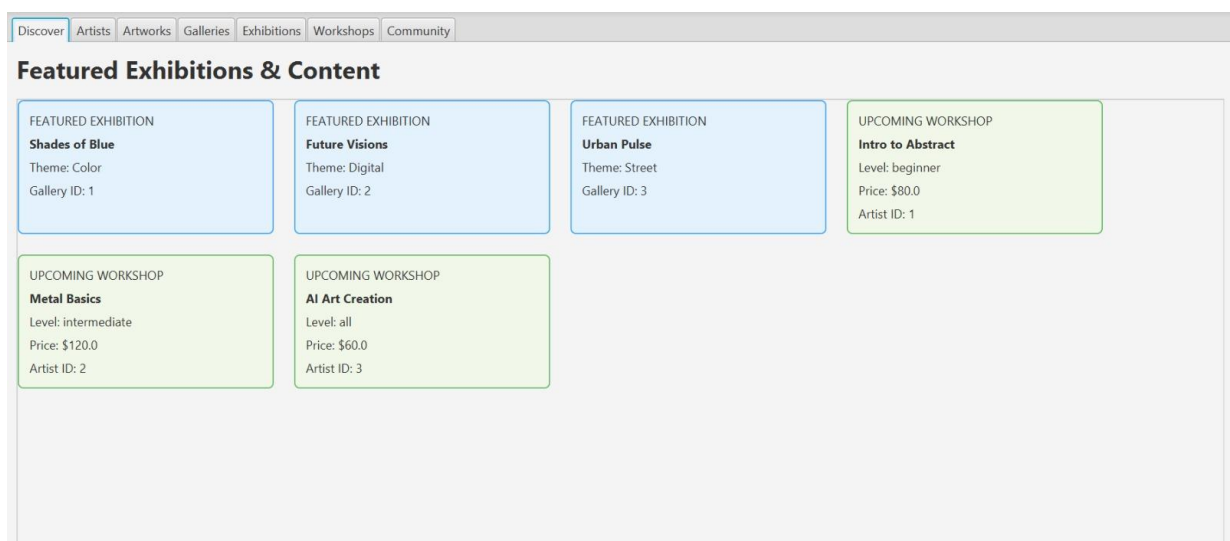


Figure 5 : Featured Exhibition page based on our database

Discover	Artists	Artworks	Galleries	Exhibitions	Workshops	Community			
Title		Artist	Type	Price	Status				
Blue Horizon		1	painting	1200.0	AVAILABLE				
Steel Rebirth		2	sculpture	3500.0	SOLD				
Neon Dreams		3	digital	600.0	AVAILABLE				
Urban Shadows		4	photo	400.0	AVAILABLE				
Clay Echoes		5	ceramic	250.0	SOLD				
Color Storm		1	painting	1500.0	RESERVED				
Glass Bloom		8	sculpture	2000.0	AVAILABLE				
AI Mirage		9	digital	700.0	AVAILABLE				
Ink Whispers		7	illustration	300.0	AVAILABLE				
Red Pulse		10	painting	1800.0	AVAILABLE				

Figure 6 : Artwork page based on our database

Discover	Artists	Artworks	Galleries	Exhibitions	Workshops	Community				
Title		Instructor		Date		Price	Level			
Intro to Abstract		1		2024-07-10T16:00		80.0	beginner			
Metal Basics		2		2024-08-05T12:00		120.0	intermediate			
AI Art Creation		3		2024-09-12T18:00		60.0	all			
Street Photo Walk		4		2024-06-20T11:00		50.0	all			
Ceramic Workshop		5		2024-05-15T16:00		70.0	beginner			
Mixed Media Lab		6		2024-07-22T13:00		90.0	intermediate			
Ink Illustration		7		2024-08-01T15:00		65.0	beginner			
Glass Art Intro		8		2024-09-05T12:00		150.0	advanced			
AI Landscapes		9		2024-10-01T17:00		60.0	all			
Experimental Painting		10		2024-11-10T11:00		100.0	intermediate			

Figure 7 : Workshop page based on our database

Gallery	Rating	Website	Address ID	
Galerie Lumière	5	https://galerie1.com	1	
ModernArt Space	4	https://gallery2.com	2	
Berlin Art Hub	5	https://gallery3.com	3	
Roma Creativa	3	https://gallery4.com	4	
Madrid Vision	4	https://gallery5.com	5	
Vienna Kunst	5	https://gallery6.com	6	
London Modern	4	https://gallery7.com	7	
Tokyo NeoArt	5	https://gallery8.com	8	
Lisbon Colors	4	https://gallery9.com	9	
Munich Studio	3	https://gallery10.com	10	

Figure 8 : Gallery page based on our database

ArtConnect

Sign in to your account

Email

Password

Enter your password

Login

No account? [Register here](#)

← Browse without account

Figure 9 : Login page of our application

Create your ArtConnect account

You will be registered as a Member.

Full Name

Email

Password

Birth Year

Phone

City

Membership Type

STANDARD ▾

Create Account

← Back to Login

Figure 10 : Creation of account page

ArtConnect Pro – Admin Panel									
Logged in as: Anna Müller								Logout	
Artists	Artworks	Galleries	Exhibitions	Workshops	Community	Review	Bookings		
Search artists...							All activity ▾	Search	Reset
Artist ID	User ID	Name	City	Email	Birth Year	Phone	Bio		
1	1	Alice Dupont	Paris	alice@example.com	1985	0600000001	Abstract painter		https://ali
2	2	Marc Leroy	Lyon	marc@example.com	1990	0600000002	Metal sculptor		https://mi
3	3	Sophie Martin	Marseille	sophie@example.com	1998	0600000003	Digital artist		https://so
4	4	John Becker	Berlin	john@example.com	1982	0600000004	Street photographer		https://jo
5	5	Emma Rossi	Rome	emma@example.com	1995	0600000005	Ceramic artist		https://em
6	6	Paul Durand	Paris	paul@example.com	2000	0600000006	Mixed media creator		https://pa
7	7	Laura Chen	Paris	laura@example.com	1997	0600000007	Illustrator		https://la
8	8	Hugo Bernard	Lille	hugo@example.com	1988	0600000008	Glass sculptor		https://hu
9	9	Nina Kovac	Zagreb	nina@example.com	1993	0600000009	AI art researcher		https://ni
10	10	Carlos Mendez	Madrid	carlos@example.com	1987	0600000010	Experimental painter		https://ca

Figure 11 : Final database view example on artist